

LUNARORBIT - PROG B

EKSAMENSPROJEKT

Gustas Pranskus

SLOTSHAVEN GYMNASIUM Mikkel Kirkby Mosthaf (MKM)

Indholdsfortegnelse

Indledning	2
Problemformulering.....	2
Systembeskrivelse	3
Trelagsmodellen.....	3
Datakilde og API.....	3
Selvskrevet klasse: MoonEngine.....	4
get_phase_name(), fra tal til tekst.....	4
format_moon_data(), samlet datatransformation	4
calculate_days_to_full_moon(), beregning	5
Klassens rolle i trelagsmodellen	5
Tekniske valg og overvejelser	5
Threading	5
Caching	5
Lokal astronomiberegning	5
Dynamisk baggrund.....	5
Brugerinteraktion og funktioner	6
Opfyldelse af eksamenskrav	6
Redegørelse for brug af AI	7
Kodeassistance	7
Kritisk vurdering.....	7
Projektforløb og tidslinje.....	7
Konklusion	8
Bilag	8

Indledning

LunarOrbit er en Python-desktop-applikation der viser månefaser, vejrdato og astronomiske tider for en valgfri dato. Applikationen er bygget som et eksamensprojekt i Programmering B og demonstrerer struktureret brug af trelagsmodellen i et grafisk brugerinterface med et galaksetema.

Formålet med projektet er at give brugeren et visuelt og intuitivt overblik over månens faser og relevante astronomiske data, og at demonstrere principperne bag data håndtering, logiklag og præsentation i en velstruktureret Python-applikation.

Problemformulering

Hvordan kan man udvikle en Python-applikation der på struktureret vis henter, transformerer og præsenterer månefase- og vejrdato for brugeren, og som opfylder kravene til trelagsmodellen, OOP og interaktivitet?

Systembeskrivelse

Trelagsmodellen

Applikationen er opbygget efter trelagsmodellen med en klar adskillelse af ansvar:

- Data laget (moon_api.py): Henter vejrdata fra Open-Meteo API og beregner månefase, solopgang, solnedgang, måneopgang og månenedgang med astronomiske algoritmer lokalt.
- Logiklaget (boilerplate.py): Indeholder klasser som MoonEngine, MoonVisuals, DateUtils og Formatters der transformerer rådata til præsenteringsklare formater.
- Præsenteringslaget (UI.py): Bygger brugergrænsefladen med CustomTkinter, tegner galaksebaggrunden med PIL og viser alle data visuelt.

Datakilde og API

Applikationen benytter to datakilder:

- Open-Meteo API: Et gratis vejr-API der leverer temperatur og skydækkeprocent. Applikationen vælger automatisk mellem forecast-endpointet (nuværende/fremtidige datoer) og arkiv-endpointet (historiske datoer).
- Lokal astronomisk beregning: Månefase, solopgang, solnedgang, måneopgang og månenedgang beregnes lokalt med NOAA-algoritmen og den julianske dagsberegning. Dette giver data for alle datoer uden API-afhængighed.

Selvskrevet klasse: MoonEngine

MoonEngine er det centrale logiklag i applikationen. Klassen fungerer som en transformationsmotor der tager rå matematiske værdier fra data laget og oversætter dem til menneskelæsbare resultater som kan vises direkte i UI'et.

Klassen bruger udelukkende statiske metoder (@staticmethod) fordi den ikke behøver at gemme nogen tilstand, den modtager input, beregner og returnerer et resultat, ligesom en matematisk funktion.

get_phase_name(), fra tal til tekst

Denne metode modtager en faseværdi mellem 0 og 1 fra datalaget og returnerer det danske navn for faser. Månens cyklus er opdelt i 8 lige store intervaller:

```
# Eksempel: phase = 0.48 giver 'Voksende gibbous'
@staticmethod
def get_phase_name(phase_value: float) -> str:
    if phase_value < 0.125:
        return "Nymåne"
    elif phase_value < 0.25:
        return "Voksende halvmåne"
    elif phase_value < 0.375:
        return "Første kvarter"
    elif phase_value < 0.5:
        return "Voksende gibbous"
    elif phase_value < 0.625:
        return "Fuldmåne"
    elif phase_value < 0.75:
        return "Aftagende gibbous"
    elif phase_value < 0.875:
        return "Sidste kvarter"
    else:
        return "Aftagende halvmåne"
```

En faseværdi på 0 svarer til nymåne (månen er ikke synlig), 0.5 svarer til fuldmåne (100% belyst), og alle værdier derimellem inddeles i de syv mellemfaser. Intervallerne er 0.125 brede fordi 1 divideret med 8 faser er 0.125.

format_moon_data(), samlet datatransformation

Denne metode er bindeledet mellem datalaget og præsentationslaget. Den modtager en rå ordbog fra MoonAPIClient og returnerer en ny ordbog med alle nødvendige displayværdier:

```
# Input fra datalaget: {"phase": 0.48, "illumination": 91.2}
# Output til UI'et:    {"phase": 0.48, "phase_name": "Voksende gibbous",
#                      "illumination": 91.2, "illumination_percent": "91%"}
@staticmethod
def format_moon_data(api_data: Dict[str, float]) -> Dict[str, any]:
    if not api_data:
        return {"phase": 0, "phase_name": "Ukendt", ...}

    illumination = api_data.get("illumination", 0)
    phase        = api_data.get("phase", 0)

    return {
        "illumination":      illumination,
        "illumination_percent": f"{int(illumination)}%",
        "phase":             phase,
        "phase_name":        MoonEngine.get_phase_name(phase)
    }
```

Metoden sørger også for fejlhåndtering: hvis `api_data` er `None` eller tom (f.eks. ved netværksfejl), returneres sikre standardværdier så applikationen ikke crasher.

calculate_days_to_full_moon(), beregning

Denne metode beregner hvor mange dage der er til næste fuldmåne baseret på den aktuelle faseværdi og månens synodiske måned (29.53 dage):

```
# Fuldmåne er ved phase = 0.5, dvs. halvvejs i cyklussen
@staticmethod
def calculate_days_to_full_moon(phase_value: float) -> float:
    days_into_cycle = phase_value * MoonEngine.SYNODIC_MONTH
    days_to_full = (0.5 * MoonEngine.SYNODIC_MONTH) - days_into_cycle

    if days_to_full < 0:
        # Fuldmåne er allerede passeret
        days_to_full += MoonEngine.SYNODIC_MONTH

    return round(days_to_full, 1)
```

Hvis `phase_value` er 0.6 (lidt efter fuldmåne), vil `days_to_full` være negativ. If-sætningen håndterer dette ved at lægge en hel månedscyklus til, så resultatet bliver dage til den næste fuldmåne.

Klassens rolle i trelagsmodellen

`MoonEngine` er et tydeligt eksempel på logiklagets opgave: den ved hverken noget om hvor data kommer fra (data laget) eller hvordan det vises (præsentationslaget). Den kun transformerer. Dette gør koden nem at teste og vedligeholde — man kan ændre UI'et uden at røre `MoonEngine`, og man kan ændre API'et uden at røre UI'et.

Tekniske valg og overvejelser

Threading

API-kald til vejr-endpointet sker i en baggrundstråd (`threading.Thread`) så brugergrænsefladen ikke fryser mens data hentes. Tkinter kræver at UI-opdateringer sker i hoved-tråden, så resultater sendes tilbage med `after(0, lambda: ...)`.

Caching

Vejrdata gemmes i en ordbog (`_weather_cache`) med datoen som nøgle. Hvis brugeren flytter slideren tilbage til en allerede hentet dato, bruges den cachede værdi i stedet for et nyt API-kald. Dette reducerer ventetid og API-belastning markant.

Lokal astronomiberegning

Frem for at hente solopgang og måneopgang fra API'et beregnes disse lokalt med astronomiske formler baseret på NOAA's algoritme. Fordelen er at beregningerne virker for alle datoer — fortid, nutid og fremtid — uden fejl fra API-grænser.

Dynamisk baggrund

Galaksebaggrunden genereres én gang i høj opløsning (1920x1080) med PIL og skaleres ved `resize-events`. Alle widget-baggrundsfarver opdateres dynamisk til at matche gradientens farve på den præcise y-position.

Brugerinteraktion og funktioner

Applikationen tilbyder følgende interaktionsmulighederne:

- Datovælger (slider): En slider med – og + ikoner lader brugeren navigere ± 60 dage fra i dag.
- Reset-knap: 'Tilbage til i dag' nulstiller slideren og henter data for dags dato.
- Realtidsur: Et ur i øverste højre hjørne opdateres hvert sekund.
- Månefase-panel: Viser fase-emoji, fasens danske navn, belysningsprocent og dage til næste fuldmåne.
- Vejrpanel: Viser min/max temperatur og skydækkeprocent for den valgte dato.
- Astronomipanel: Viser solopgang, solnedgang, måneopgang, månenedgang, UV-indeks og nedbørssandsynlighed.
- Fuldskærm: Applikationen skalerer baggrund og paneler korrekt ved alle vinduesstørrelser.

Opfyldelse af eksamenskrav

Krav	Opfyldelse i LunarOrbit
Datakilde (API/fil)	Open-Meteo REST API til vejr. Lokal astronomisk beregning til månefase og tider.
Logiktransformation	MoonEngine transformerer rå faseværdier (0-1) til emoji, dansk navn og dage til fuldmåne. DateUtils konverterer slider-position til dato.
OOP / selvskrevne klasser	1 selvskreven klasse: MoonEngine,
Interaktivitet	Slider til datovælg, reset-knap, realtidsur. Brugeren kan navigere frit i tid.
Struktureret præsentation	To informationspaneler med tydelige overskrifter og emoji-ikoner. Galaksebaggrund med dynamisk skalering.
Inputvalidering	slider_value_to_date() begrænser værdien til 0-100. Datoformat valideres med datetime.strptime(). API-fejl håndteres med try/except.
Velkommenteret kode	Alle klasser og metoder har docstrings med beskrivelse, Args og Returns.
Versionering	Kode lagret i GitHub-repository.

Redegørelse for brug af AI

AI (Claude, Anthropic) er anvendt som et aktivt udviklingsværktøj gennem hele projektet. Brugen kan beskrives i følgende kategorier:

Kodeassistance

AI er brugt til at debugge fejl, foreslå løsninger på tekniske problemer og forklare Python-koncepter. Konkrete eksempler inkluderer:

- Fejlfinding i threading-implementationen (after(0, lambda) mønstret for Tkinter-trådsikkerhed).
- Implementering af NOAA's solopgangs algoritme og den julianske dagsberegning.
- Løsning af CustomTkinter-specifikke problemer med transparent widget-baggrunde.
- Rettelse af fejl i API-parameternavne og valg af korrekt endpoint for historiske datoer.

Kritisk vurdering

Ikke al AI-genereret kode er accepteret ukritisk. Eksempler på fejl der er identificeret og rettet: negativ ekliptikkens hældning i solopgangsformlen der vendte sommer og vinter om, CTKCanvas der ikke eksisterer i CustomTkinter, og resize-events der trigger fra alle child-widgets i stedet for kun vinduet. Disse fejl er fundet ved at teste applikationen og sammenligne med forventede resultater.

Projektforløb og tidslinje

Projektet er udført selvstændigt over ca. 5-6 uger. Nedenfor er en oversigt over arbejdsforløbet:

Uge	Aktivitet
Uge 1	Ideudvikling og konceptafklaring. Valg af tema (månefaser), aflevering af projektbeskrivelse d. 27/2. Opsætning af udviklingsmiljø med Python, CustomTkinter og virtuel environment.
Uge 2	Opbygning af grundstruktur efter trelagsmodellen. Implementering af MoonAPIClient med månefaseberegning og første API-kald til Open-Meteo. Oprettelse af GitHub-repository.
Uge 3	Udvikling af logiklaget: MoonEngine, MoonVisuals, DateUtils og Formatters klasser. Implementering af format_moon_data(), get_phase_name() og calculate_days_to_full_moon().
Uge 4	Opbygning af UI med CustomTkinter. Galaksebaggrund med PIL-gradient og stjerner. Fejlretning af slider (forkert start/slut-dato), 3 måne-emojis og performance-problemer med threading og caching.
Uge 5	Astronomipanel tilføjet. Implementering af lokal solopgang/måneopgang beregning med NOAA-algoritmen (erstatning for fejlende API-parametre). Rettelse af sommer/vinter-fejl i formelen.
Uge 6	Finpudsning af UI: dynamisk baggrundsskalering, fjernelse af mørke bokse bag labels, korrekt panellayout. Kodegennemgang, kommentering og docstrings. Skrivning af synopsis.

Konklusion

LunarOrbit demonstrerer en velfungerende implementering af trelagsmodellen i en Python-desktop-applikation. Applikationen henter og transformerer data fra et eksternt API og lokale astronomiske beregninger, præsenterer det visuelt med et galaksetema, og giver brugeren en intuitiv interaktionsoplevelse via datosliders og realtidsdata.

Projektet har givet praktisk erfaring med OOP, API-integration, threading, PIL-billedbehandling og CustomTkinter. AI har været et nyttigt værktøj der har accelereret udviklingen, men har krævet kritisk stillingtagen til de løsninger der er foreslået.

Alle ikke-funktionelle og funktionelle krav fra opgavebeskrivelsen er opfyldt, jf. afsnit 7.

Bilag

Opgavens program LunarOrbit

https://github.com/Gu5ta5/Eksamensprojekt_LunarOrbit